Slack → DuckDB team

Hey folks — we're finishing up a GPU-native scan path for Sirius (our DuckDB GPU extension) that decodes `.duckdb` block storage directly on the GPU, bypassing the `duckdb_scan_task` table-function path. Results are good: **TPC-H SF=100 warm on GH200 is now 1.33× faster than DuckDB CPU**, 15/22 queries win. We'd like to land the branch cleanly and have two asks — one easy and public, one trickier and scoped to read-only use.

Ask #1 — Five tiny public accessors (should be uncontroversial)

To scan DuckDB blocks at the block level from the extension side, we need five getters that expose state already computed privately. Zero behavior change, pure declarations in headers. All under ~3 lines each.

Header	Accessor	What it exposes
`storage/data_table.hpp`	`GetRowGroupCollectionRef()`	`shared_ptr<RowGroupCollection>&` (the already-private `row_groups`)
`storage/table/row_group_collection.hpp`	`GetRowGroupsDirect()`	the segment tree (today's `GetRowGroups()` is protected)
`storage/table/row_group.hpp`	`GetColumnDirect(StorageIndex)`	`ColumnData&` (today's `GetColumn` protected)
`storage/table/column_segment.hpp`	`GetCompressionType()`	wraps `function.get().type`
`storage/table/standard_column_data.hpp`	`GetValidityData()`	the already-private validity ColumnData

Sketch (this is all of them combined):

```
// data_table.hpp
shared_ptr<RowGroupCollection>& GetRowGroupCollectionRef() { return row_groups; }

// row_group_collection.hpp
shared_ptr<RowGroupSegmentTree> GetRowGroupsDirect() const { return GetRowGroups(); }

// row_group.hpp
ColumnData& GetColumnDirect(const StorageIndex &c) const { return GetColumn(c); }

// column_segment.hpp
CompressionType GetCompressionType() const { return function.get().type; }
```

```
// standard_column_data.hpp
ValidityColumnData& GetValidityData() { return *validity; }
```

Why: without these, the extension can't walk the segment tree or discover compression types without duplicating DuckDB's internal logic. We've been carrying these as a local patch for a few months — time to upstream them so the extension stops needing a fork of the DuckDB submodule.

Impact to you: none. These add read accessors, don't touch write paths, can't be used to corrupt state (the returned refs are const-walked by us). We're happy to open the PR, write tests, etc.

Ask #2 — A `GetDirectBlockPointer(block_id)` on `BlockManager` for read-only `.duckdb` files

This one is trickier and we want to talk through it before opening a PR.

The problem

`BufferManager::Pin()` dominates our GPU scan wall clock. On TPC-H SF=100 Q1 (lineitem, 1 column), we measured:

Component	Time	%
GPU decode kernels	7ms	1.1%
H2D transfer	29ms	4.6%
`Pin()` overhead	**470ms**	**74.6%**
Other CPU	124ms	19.7%
Wall	630ms	

At ~29K segments × ~16µs per `Pin()`, we pay ~470ms in a path that, for a read-only mmap'd file, is a no-op the mmap could satisfy directly. The mutex is only ~5–10% of per-segment cost — the bigger cost is the refcount work + segment-tree traversal that `Pin()` does for writeability guarantees we don't need.

What we actually need

A way to get the mmap'd pointer for block data with **no lock and no refcount bookkeeping**, guarded to read-only databases so we can't observe stale or evicted pages:

```
// duckdb/storage/block_manager.hpp
virtual data_ptr_t GetDirectBlockPointer(block_id_t block_id) { return nullptr; }

// SingleFileBlockManager overrides it: lazy mmap behind atomic double-checked
// locking, returns base + BLOCK_START + block_id * alloc_size + header.
// Guarded on options.read_only && !encryption_enabled. Null for everyone else.
```

We prototyped this (5-commit stack, ~120 LOC) and it works: TPC-H SF=100 cold/ query drops from **16s → ~4–5s** when we avoid prefaulting.

Why we can't just do it ourselves

We **can**, and we currently do — our extension-side fallback computes the same offset math from `BLOCK_START = 4096*3 + block_id * alloc_size + header_size` and validates once at startup against a `Pin()`'d block. Pure Sirius version is within ~2–3% of the DuckDB-API version's perf.

But:

1. **It's a layout guess.** Any change to DuckDB's file layout silently breaks us. We catch it in our `std::call_once` validator but only after the first block; a lurking subtle change (e.g. encryption header reshuffle) could slip through.
2. **We're reaching into internals.** Reading `segment.block->GetBlockAllocSize()` and `GetBlockHeaderSize()` from ColumnSegment is fine, but the offset of the first block inside the file (`FILE_HEADER_SIZE * 3`) is a storage-format constant we currently re-derive.
3. **Upstream is where correctness lives.** A 6-line `GetDirectBlockPointer` on `SingleFileBlockManager` centralizes the format-knowledge in DuckDB and gives extensions a guaranteed-correct escape hatch.

What we're NOT asking for

- Not asking to change `Pin()`. That's the hot write path, we don't want to touch it.
- Not asking for a buffer-pool bypass on writable DBs. The read-only gate is the whole safety story.
- Not asking for encryption support — `encryption_enabled` short-circuits to null and we fall back to `Pin()`.

What we'd do if you say yes

Open a PR with: the virtual accessor default-implemented as `return nullptr` on base `BlockManager`, the override on `SingleFileBlockManager`, tests, and a docs note that read-only extension readers can use it to avoid Pin overhead. Gated behind `options.read_only && !encryption_enabled`. ~150 LOC, 5 files.

If you'd rather we keep the layout-guess approach Sirius-side, we can — we just want to flag the maintainability cost and get a nod either way before we finalize the extension PR.

Happy to jump on a call for #2. #1 we'll open a PR for and ping you unless anyone objects.